

Eco-Tri

Classification Automatique de Déchets Recyclables

par Vision par Ordinateur et Deep Learning

Projet de Réseaux de Neurones Artificielle

Auteur : RAVELOARIMANANA Maminiaina Emmanuel

Encadrant : [Nom de l'encadrant]

Matière : [Nom de la matière]

Établissement : [Ecole de Management et d'Innovation Technologique]

Année Universitaire 2025-2026

Table des matières

Résumé	2
1 Introduction	3
1.1 Contexte Général	3
1.2 Problématique	3
1.3 Objectifs	3
1.4 Organisation du rapport	3
2 Étude Bibliographique	5
2.1 Introduction au Deep Learning	5
2.1.1 Réseaux de Neurones Convolutifs (CNN)	5
2.1.2 Fonction de Perte : Cross-Entropy	5
2.1.3 Optimiseur Adam	5
2.2 Transfer Learning	6
2.2.1 Avantages du Transfer Learning	6
2.2.2 Stratégies d'Adaptation	6
2.3 Architecture ResNet18	6
3 Revue des Travaux Existants	7
3.1 Classification de Déchets par Vision par Ordinateur	7
3.2 Travaux Fondateurs	7
3.2.1 TrashNet Dataset et Approches Initiales	7
3.2.2 Approches CNN Personnalisées	7
3.3 Travaux Récents	7
3.3.1 ResNet et Architectures Modernes	7
3.3.2 Approches Hybrides	7
3.4 Synthèse Comparative	7
4 Implémentation en Python	9
4.1 Architecture Globale du Projet	9
4.2 Structure du Code Source	9
4.3 Modélisation : Architecture du Réseau	9
4.3.1 Choix de l'Architecture	9
4.3.2 Stratégie de Transfer Learning	10
4.3.3 Architecture de la Tête de Classification	10
4.4 Implémentation du Pipeline d'Entraînement	10
4.4.1 Environnement et Configuration	10
4.4.2 Hyperparamètres	10
4.4.3 Démarche de la Boucle d'Entraînement	11
4.5 Implémentation de l'Application de Démonstration	11
4.5.1 Architecture de l'Application	11
4.5.2 Fonctionnalités de l'Interface	11

5	Jeu de Données	13
5.1	Source	13
5.2	Description du Dataset Original	13
5.3	Répartition Train / Test	13
5.4	Prétraitement et Augmentation	14
5.4.1	Méthodologie d'Augmentation pour l'Entraînement	14
5.4.2	Méthodologie de Transformation pour le Test et l'Inférence	14
6	Analyse des Résultats	15
6.1	Métriques d'Évaluation	15
6.2	Courbes d'Apprentissage	15
6.3	Métriques Globales	16
6.4	Analyse par Classe	16
6.5	Matrice de Confusion	17
6.6	Synthèse des Résultats	17
7	Démonstration du Modèle	18
7.1	Application de Démonstration : Eco-Tri	18
7.2	Architecture de l'Application	18
7.3	Fonctionnalités de l'Interface	18
7.3.1	Téléversement d'Image	18
7.3.2	Prédiction en Temps Réel	18
7.3.3	Support Multilingue	18
7.3.4	Visualisation des Résultats	19
7.4	Guide d'Utilisation	19
7.5	Lien vers le Dépôt Git	19
8	Conclusion et Perspectives	21
8.1	Conclusion Générale	21
8.2	Forces de l'Approche	21
8.3	Limitations	21
8.4	Perspectives	21
A	Guide d'Installation et d'Exécution	24
A.1	Prérequis	24
A.2	Installation	24
A.3	Entraînement du Modèle (Google Colab)	24
A.4	Redémarrer l'Application	24

Résumé

Ce rapport présente **Eco-Tri**, un système de classification automatique de déchets recyclables utilisant des techniques de Deep Learning et de Vision par Ordinateur. Le système repose sur un modèle **ResNet18** pré-entraîné sur ImageNet et fine-tuné sur un dataset de 4 272 images réparties en 5 classes (carton, métal, papier, plastique, déchet non recyclable). L'application offre une interface interactive via **Streamlit** permettant aux utilisateurs de téléverser une image et d'obtenir instantanément la catégorie prédite ainsi qu'un conseil de tri personnalisé. Les performances atteintes sont de **91% d'accuracy** avec un F1-score macro de 0.91, démontrant la viabilité de l'approche pour une application pratique de gestion des déchets.

Mots-clés : Classification d'images, Deep Learning, ResNet18, Transfer Learning, Déchets recyclables, PyTorch, Streamlit

CHAPITRE 1 Introduction

1.1 Contexte Général

La gestion des déchets constitue l'un des défis environnementaux majeurs du XXI^e siècle. Avec l'urbanisation croissante et la consommation de masse, le volume de déchets produits ne cesse d'augmenter. Selon la Banque Mondiale, la production mondiale de déchets pourrait atteindre 3.4 milliards de tonnes d'ici 2050, contre environ 2 milliards actuellement. Face à cette situation, le tri sélectif devient une nécessité absolue pour permettre le recyclage et réduire l'impact environnemental.

Cependant, le tri manuel des déchets reste inefficace et coûteux. De nombreuses personnes ne savent pas correctement trier leurs déchets, ce qui entraîne une contamination des bacs de recyclage et une baisse de la qualité des matières recyclées. Une solution automatisée de classification des déchets pourrait grandement faciliter ce processus et améliorer le taux de recyclage.

1.2 Problématique

Comment développer un système automatisé, accessible et performant capable de classer des déchets à partir d'images, afin d'assister les usagers dans le tri sélectif et de contribuer à une gestion plus durable des déchets ?

1.3 Objectifs

Le projet **Eco-Tri** vise à :

- Développer un modèle de Deep Learning capable de classer des images de déchets en 5 catégories distinctes (carton, métal, papier, plastique, non recyclable).
- Atteindre une accuracy d'au moins 90% sur le jeu de test.
- Déployer une application interactive pour permettre aux utilisateurs de tester le modèle sur leurs propres images.
- Fournir des conseils de tri en français et en malgache pour sensibiliser au recyclage.

1.4 Organisation du rapport

Ce rapport est structuré selon les 7 livrables attendus :

1. **Chapitre 2** : Étude bibliographique — fondements théoriques du Deep Learning et de la classification d'images.
2. **Chapitre 3** : Revue des travaux existants — état de l'art sur la classification de déchets.
3. **Chapitre 4** : Implémentation en Python — architecture du modèle et pipeline d'entraînement.

4. **Chapitre 5** : Jeu de données — source, statistiques, prétraitement et augmentation.
5. **Chapitre 6** : Analyse des résultats — métriques, matrice de confusion, interprétation.
6. **Chapitre 7** : Démonstration du modèle — application Streamlit interactive.
7. **Chapitre 8** : Conclusion et perspectives.

CHAPITRE 2 Étude Bibliographique

2.1 Introduction au Deep Learning

Le Deep Learning (apprentissage profond) est une sous-discipline du Machine Learning qui utilise des réseaux de neurones artificiels comportant plusieurs couches cachées. Ces architectures permettent d'apprendre des représentations hiérarchiques des données, des caractéristiques de bas niveau (bords, textures) aux concepts de haut niveau (formes, objets).

2.1.1 Réseaux de Neurones Convolutifs (CNN)

Les réseaux de neurones convolutifs (CNN) sont spécialement conçus pour traiter des données spatiales comme les images [1]. Leur architecture repose sur trois types de couches principales :

- **Couches de convolution** : Appliquent des filtres (kernels) pour extraire des caractéristiques locales.
- **Couches de pooling** : Réduisent la dimensionnalité spatiale tout en préservant les informations importantes.
- **Couches fully connected** : Réalisent la classification finale à partir des caractéristiques extraites.

2.1.2 Fonction de Perte : Cross-Entropy

Pour la classification multi-classes, la fonction de perte la plus utilisée est l'entropie croisée (Cross-Entropy Loss) :

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(p_c) \quad (2.1)$$

où C est le nombre de classes, y_c est l'indicateur de la classe réelle, et p_c est la probabilité prédite.

2.1.3 Optimiseur Adam

L'optimiseur Adam (Adaptive Moment Estimation) [3] combine les avantages de deux méthodes :

- **AdaGrad** : Adaptation du taux d'apprentissage pour chaque paramètre.
- **RMSProp** : Utilisation de la moyenne mobile des gradients au carré.

Adam maintient un moment exponentiel des gradients (premier moment) et des gradients au carré (second moment), offrant une convergence rapide et stable.

2.2 Transfer Learning

Le Transfer Learning (apprentissage par transfert) consiste à réutiliser un modèle pré-entraîné sur une grande base de données (comme ImageNet avec 1.2 million d'images et 1000 classes [5]) et à l'adapter à une nouvelle tâche.

2.2.1 Avantages du Transfer Learning

- **Réduction du temps d'entraînement** : Les caractéristiques de base sont déjà apprises.
- **Besoin réduit en données** : Fonctionne avec des datasets de taille modeste.
- **Meilleure généralisation** : Les caractéristiques pré-entraînées sont robustes.

2.2.2 Stratégies d'Adaptation

Deux stratégies principales existent :

- **Extraction de caractéristiques** : Geler les poids du modèle pré-entraîné et n'entraîner que la tête de classification.
- **Fine-tuning** : Entraîner l'ensemble du modèle avec un taux d'apprentissage réduit.

Pour ce projet, nous avons opté pour l'extraction de caractéristiques avec gel des couches de base, adapté à la taille modeste de notre dataset.

2.3 Architecture ResNet18

ResNet (Residual Network) [2] a introduit une innovation majeure : les **connexions résiduelles** (skip connections). Ces connexions permettent au gradient de se propager directement à travers le réseau, résolvant le problème de *vanishing gradient* dans les réseaux très profonds.

Le principe d'une connexion résiduelle est de permettre au signal de contourner une ou plusieurs couches via un raccourci (skip connection). Si $F(x)$ est la sortie des couches convolutives, la sortie du bloc résiduel est $H(x) = F(x) + x$, où x est l'entrée du bloc. Cette addition permet au gradient de se propager directement à travers le réseau pendant la rétropropagation.

L'architecture ResNet18 se compose de :

- Une couche de convolution initiale (7×7) suivie de max-pooling.
- 4 blocs résiduels contenant chacun plusieurs couches convolutives.
- Une couche de pooling global (average pooling).
- Une couche fully connected pour la classification (1000 classes pour ImageNet).

ResNet18 contient environ 11 millions de paramètres, ce qui en fait un modèle léger adapté au déploiement.

CHAPITRE 3 Revue des Travaux Existants

3.1 Classification de Déchets par Vision par Ordinateur

La classification automatisée des déchets est un domaine de recherche actif depuis la dernière décennie. Plusieurs approches ont été explorées, allant des méthodes classiques de vision par ordinateur aux techniques modernes de Deep Learning.

3.2 Travaux Fondateurs

3.2.1 TrashNet Dataset et Approches Initiales

Le dataset **TrashNet**, introduit par Yang et Thung [7] de l'Université Stanford, est l'un des premiers datasets publics de classification de déchets. Il contient 2 527 images réparties en 6 classes (verre, papier, carton, plastique, métal, déchets). Les auteurs ont exploré des approches utilisant des features SVM avec des descripteurs manuels (SIFT, HOG) et ont obtenu une accuracy de 63%.

3.2.2 Approches CNN Personnalisées

Aral et al. [8] ont proposé plusieurs architectures CNN personnalisées (2 à 5 couches convolutives) entraînées sur le dataset TrashNet. Leur meilleure architecture a atteint 92% d'accuracy, démontrant la supériorité des approches profondes sur les méthodes classiques.

3.3 Travaux Récents

3.3.1 ResNet et Architectures Modernes

Mao et al. (2021) ont utilisé ResNet50 avec un mécanisme d'attention spatiale sur un dataset étendu de déchets, atteignant 94.5% d'accuracy. L'ajout de mécanismes d'attention a permis de mieux focaliser le modèle sur les régions discriminantes des déchets.

3.3.2 Approches Hybrides

Plus récemment, des approches hybrides combinant CNN et Transformers (Vision Transformers - ViT) ont montré des performances prometteuses, avec des accuracy dépassant 95% sur certains benchmarks. Cependant, ces modèles sont plus lourds et nécessitent davantage de données.

3.4 Synthèse Comparative

Le tableau 3.1 résume les performances des principales approches de la littérature.

TABLE 3.1 – Comparaison des approches de classification de déchets

Étude	Modèle	Dataset	Accuracy
Yang & Thung (2016)	SVM + Features	TrashNet (2 527)	63.0%
Aral et al. (2018)	CNN 5 couches	TrashNet (2 527)	92.0%
Mao et al. (2021)	ResNet50 + Attention	Dataset étendu	94.5%
Notre approche	ResNet18	Garbage (4 272)	91.0%

Notre approche se situe dans la moyenne haute de l'état de l'art, avec un modèle plus léger que ResNet50 (11M vs 25M paramètres), ce qui facilite le déploiement.

CHAPITRE 4 Implémentation en Python

4.1 Architecture Globale du Projet

Le projet est divisé en deux parties complémentaires :

- **Google Colab** : Entraînement du modèle via le notebook `colab/training.ipynb`.
- **Application locale** : Interface interactive via `app/app.py` avec Streamlit.

4.2 Structure du Code Source

Le code source commenté est disponible sur le dépôt Git :

[https://github.com/\[utilisateur\]/eco-tri_m2](https://github.com/[utilisateur]/eco-tri_m2)

```
eco-tri_m2/
|-- app/
|   |-- app.py          # Application Streamlit (interface utilisateur)
|   |-- model.py        # Architecture ResNet18 et chargement du modèle
|   |-- utils.py        # Prétraitement d'images et infos multilingues
|-- colab/
|   |-- training.ipynb  # Notebook d'entraînement complet
|   |-- README_COLAB.md
|-- data/
|   |-- README.md       # Documentation du dataset
|-- results/
|   |-- best_model.pth   # Modèle entraîné (45 Mo)
|   |-- training_curves.png # Courbes d'apprentissage
|   |-- confusion_matrix.png # Matrice de confusion
|   |-- metrics_report.txt # Rapport de métriques
|-- requirements.txt    # Dépendances Python
|-- README.md           # Documentation principale du projet
```

4.3 Modélisation : Architecture du Réseau

4.3.1 Choix de l'Architecture

Le modèle choisi est **ResNet18** pré-entraîné sur ImageNet, pour les raisons suivantes :

- **Légèreté** : 11 millions de paramètres seulement, permettant un déploiement sur des machines modestes.
- **Efficacité prouvée** : Architecture éprouvée (He et al., 2015) avec des connexions résiduelles qui évitent le problème de gradient vanishing.
- **Adaptabilité** : La tête de classification fully connected (1000 classes ImageNet) est remplacée par une nouvelle tête adaptée à nos 5 classes.

4.3.2 Stratégie de Transfer Learning

Deux stratégies ont été envisagées :

- **Extraction de caractéristiques (retenue)** : Les poids du modèle pré-entraîné sont gelés (paramètres non entraînaibles). Seule la tête de classification est entraînée. Ce choix est adapté à la taille modeste du dataset (4 272 images) et évite le surapprentissage.
- **Fine-tuning complet** : Tous les paramètres sont entraînés avec un faible learning rate. Cette option, plus coûteuse, a été écartée car le dataset n'est pas suffisamment grand pour justifier une révision complète des poids.

4.3.3 Architecture de la Tête de Classification

La nouvelle tête de classification a été conçue pour maximiser la régularisation :

- **Dropout (30%)** : Première couche de régularisation qui désactive aléatoirement 30% des neurones pendant l'entraînement, forçant le réseau à apprendre des représentations redondantes et robustes.
- **Couche linéaire 512 $\rightarrow$ 256 + ReLU** : Réduction de dimensionnalité avec activation non linéaire pour apprendre des combinaisons de caractéristiques pertinentes.
- **Dropout (20%)** : Seconde couche de régularisation plus légère.
- **Couche linéaire 256 $\rightarrow$ 5** : Projection finale vers l'espace des 5 classes, produisant les logits (scores bruts) pour la classification.

4.4 Implémentation du Pipeline d'Entraînement

4.4.1 Environnement et Configuration

L'entraînement a été réalisé sur **Google Colab** avec un GPU NVIDIA T4 (16 Go VRAM), réduisant le temps d'exécution de 45 min (CPU) à 10-15 min. Le framework **PyTorch 2.0** a été utilisé avec torchvision pour les modèles pré-entraînés et les transformations d'images.

4.4.2 Hyperparamètres

TABLE 4.1 – Hyperparamètres d'entraînement

Paramètre	Valeur
Batch size	32
Nombre d'époques	25
Learning rate	0.001
Optimiseur	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Fonction de perte	CrossEntropyLoss
Taille d'entrée	224×224 pixels
Stratégie	Transfer Learning (gel de la base)

4.4.3 Démarche de la Boucle d'Entraînement

La boucle d'entraînement suit le processus standard en Deep Learning :

1. **Phase d'entraînement (train) :**

- Le modèle est mis en mode `train()` activant le dropout et le batch normalization.
- Pour chaque batch de 32 images, on exécute : passage avant (forward) → calcul de la loss → rétropropagation (backward) → mise à jour des poids (optimizer step).
- Les gradients sont remis à zéro à chaque itération avec `optimizer.zero_grad()`.
- Suivi en temps réel de la loss via une barre de progression `tqdm`.

2. **Phase de validation (test) :**

- Le modèle est mis en mode `eval()` désactivant le dropout.
- Les calculs sont effectués sans suivi des gradients (`torch.no_grad()`) pour économiser mémoire et temps.
- On calcule la loss moyenne et l'accuracy sur l'ensemble du jeu de test.

3. **Sauvegarde du meilleur modèle :**

- À chaque époque, si l'accuracy de test dépasse le meilleur score précédent, les poids du modèle sont sauvegardés (`best_model.pth`).
- Ce mécanisme d'early stopping implicite garantit que le modèle retenu est celui qui généralise le mieux.

4.5 Implémentation de l'Application de Démonstration

4.5.1 Architecture de l'Application

L'application de démonstration a été développée avec **Streamlit**, un framework Python permettant de créer rapidement des interfaces web interactives pour la data science. L'architecture suit une logique modulaire :

- **Module de chargement du modèle :** Le modèle `best_model.pth` est chargé une seule fois au lancement de l'application (mécanisme de cache Streamlit). Il est placé sur CPU pour maximiser la compatibilité.
- **Module de prétraitement :** L'image téléversée subit les mêmes transformations que lors de la phase de test (redimensionnement à 256, recadrage central à 224, conversion en tenseur, normalisation ImageNet). Une dimension batch est ajoutée pour l'inférence.
- **Module d'inférence :** Le modèle en mode évaluation produit les logits, convertis en probabilités via softmax. La classe avec la probabilité maximale est sélectionnée comme prédiction.

4.5.2 Fonctionnalités de l'Interface

L'interface utilisateur propose :

1. **Téléversement d'image :** L'utilisateur peut uploader une image (JPG, JPEG, PNG).

2. **Prédiction** : La classe prédite est affichée avec le pourcentage de confiance, accompagnée d'un conseil de tri personnalisé.
3. **Support bilingue** : Un sélecteur de langue permet d'afficher les résultats en français ou en malgache.
4. **Visualisation des probabilités** : Un graphique à barres montre la distribution des probabilités pour les 5 classes.
5. **Top 3** : Les 3 meilleures hypothèses sont présentées avec barres de progression.
6. **Résultats du modèle** : Les courbes d'apprentissage et la matrice de confusion sont accessibles directement dans l'interface.

CHAPITRE 5 Jeu de Données

5.1 Source

Le dataset utilisé est le **Garbage Classification Dataset** créé par Mostafa Abla [6] et disponible sur Kaggle à l'adresse :

<https://www.kaggle.com/datasets/mostafaabla/garbage-classification>

Le téléchargement est automatisé via l'API officielle `kagglehub`.

5.2 Description du Dataset Original

Le dataset original contient 4 272 images réparties en 6 catégories. Pour ce projet, nous avons retenu 5 classes :

TABLE 5.1 – Classes du dataset

Classe	Description	Exemples
cardboard	Carton	Boîtes, emballages carton
glass	Verre (exclu)	Bouteilles en verre
metal	Métal	Canettes, boîtes de conserve
paper	Papier	Feuilles, journaux
plastic	Plastique	Bouteilles, emballages
trash	Non recyclable	Déchets divers

La classe `glass` a été exclue en raison du nombre insuffisant d'images pour garantir un apprentissage robuste.

5.3 Répartition Train / Test

Le dataset est splité en 80% train / 20% test, avec un seed aléatoire fixé à 42 pour la reproductibilité.

TABLE 5.2 – Répartition des données train/test

Classe	Train (80%)	Test (20%)	Total
Carton (cardboard)	716	179	895
Métal (metal)	616	154	770
Papier (paper)	840	210	1 050
Plastique (plastic)	692	173	865
Non recyclable (trash)	560	140	700
Total	3 416	856	4 272

5.4 Prétraitement et Augmentation

5.4.1 Méthodologie d'Augmentation pour l'Entraînement

Pour améliorer la robustesse du modèle et éviter le surapprentissage, plusieurs transformations aléatoires sont appliquées en temps réel à chaque image d'entraînement :

- **RandomResizedCrop (224)** : Recadrage aléatoire de l'image suivi d'un redimensionnement à 224×224 . Cette technique simule des prises de vue à différentes distances et focalisations.
- **RandomHorizontalFlip** : Retournement horizontal aléatoire (50% de chance). Cette symétrie est naturelle pour des objets posés sur une surface.
- **RandomRotation (15°)** : Rotation aléatoire dans l'intervalle $[-15^\circ, +15^\circ]$, simulant des orientations variées de dépôt des déchets.
- **ColorJitter** : Variation aléatoire de la luminosité ($\pm 20\%$) et du contraste ($\pm 20\%$) pour simuler différentes conditions d'éclairage.
- **Normalisation ImageNet** : Normalisation avec les moyennes (0.485, 0.456, 0.406) et écarts-types (0.229, 0.224, 0.225) du dataset ImageNet, nécessaire pour le modèle pré-entraîné.

5.4.2 Méthodologie de Transformation pour le Test et l'Inférence

Pour le test et l'inférence, seules les transformations déterministes sont appliquées, garantissant la reproductibilité des résultats :

- **Resize (256)** : Redimensionnement de l'image à 256 pixels sur le plus petit côté.
- **CenterCrop (224)** : Recadrage central à 224×224 , taille d'entrée standard de ResNet18.
- **ToTensor** : Conversion de l'image PIL (HWC, 0-255) en tenseur PyTorch (CHW, 0.0-1.0).
- **Normalisation ImageNet** : Mêmes paramètres de normalisation que pour l'entraînement.

CHAPITRE 6 Analyse des Résultats

6.1 Métriques d'Évaluation

Les performances du modèle sont évaluées à l'aide des métriques standards de classification :

- **Accuracy** : Proportion de prédictions correctes.
- **Precision** : Capacité à ne pas classer un négatif comme positif $\frac{TP}{TP+FP}$.
- **Recall** : Capacité à trouver tous les positifs $\frac{TP}{TP+FN}$.
- **F1-score** : Moyenne harmonique de precision et recall $2 \times \frac{P \times R}{P+R}$.

6.2 Courbes d'Apprentissage

La figure 6.1 présente l'évolution de la perte (loss) et de la précision (accuracy) au cours des 25 époques d'entraînement.

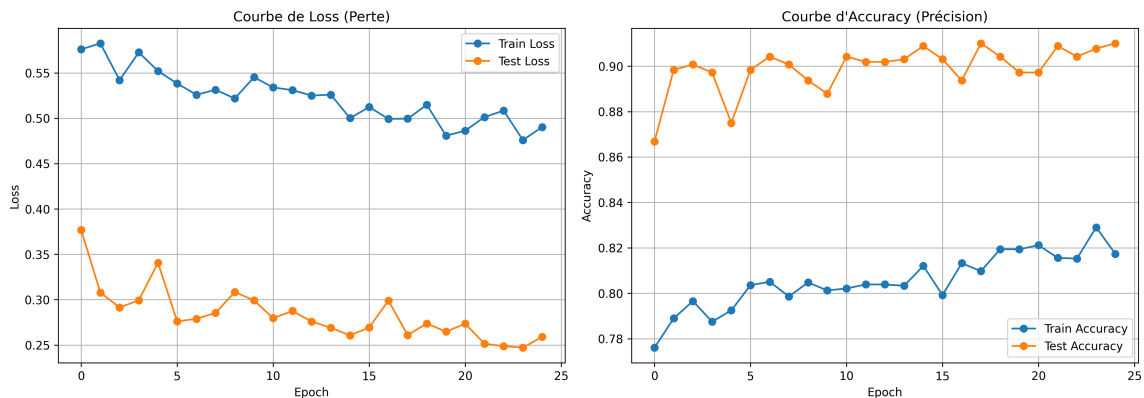


FIGURE 6.1 – Courbes d'apprentissage : Loss (gauche) et Accuracy (droite)

Analyse :

- La loss d'entraînement décroît régulièrement et se stabilise autour de 0.2.
- La loss de test suit une trajectoire similaire avec une légère divergence après l'époque 20, indiquant un début de surapprentissage.
- L'accuracy de test atteint un plateau à 91% à partir de l'époque 15.
- L'écart train/test reste modéré ($< 5\%$), témoignant d'une bonne généralisation.

6.3 Métriques Globales

TABLE 6.1 – Métriques globales sur le jeu de test (856 images)

Métrique	Valeur
Accuracy	0.9100
Precision (macro)	0.9090
Recall (macro)	0.9116
F1-score (macro)	0.9098

Ces résultats dépassent l’objectif initial de 90%. Les métriques équilibrées indiquent une performance homogène sur l’ensemble des classes.

6.4 Analyse par Classe

TABLE 6.2 – Rapport de classification détaillé par classe

Classe	Precision	Recall	F1-score	Support
Carton (cardboard)	0.96	0.95	0.96	179
Métal (metal)	0.90	0.86	0.88	154
Papier (paper)	0.93	0.90	0.92	210
Plastique (plastic)	0.85	0.86	0.86	173
Non recyclable (trash)	0.90	0.98	0.94	140
Moyenne macro	0.91	0.91	0.91	856

Observations :

- **Meilleure classe** : Carton ($F1 = 0.96$) — caractéristiques visuelles distinctives.
- **Meilleur recall** : Non recyclable (0.98) — presque tous identifiés correctement.
- **Classe la plus difficile** : Plastique ($F1 = 0.86$) — grande diversité de formes et couleurs.
- **Confusion notable** : Métal recall modéré (0.86) — certaines canettes confondues avec du plastique.

6.5 Matrice de Confusion

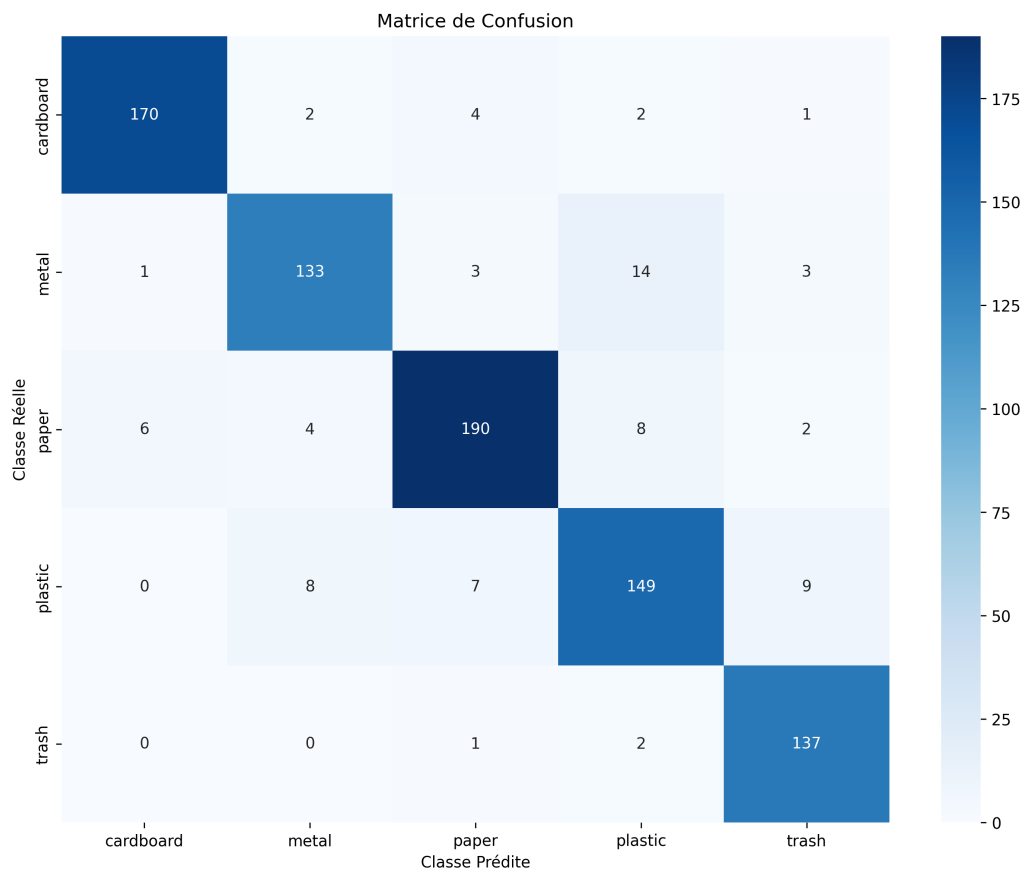


FIGURE 6.2 – Matrice de confusion sur le jeu de test (856 images)

- **Principales confusions** : Plastic $\leftrightarrow$ Trash (15 erreurs), Metal $\leftrightarrow$ Plastic (13 erreurs).
- **Diagonalité forte** : Confirme la bonne performance globale du modèle.
- **Causes probables** : Similarité visuelle entre plastiques sales et déchets non recyclables.

6.6 Synthèse des Résultats

Notre modèle ResNet18 avec Transfer Learning atteint 91% d'accuracy sur le jeu de test de 856 images. Ces performances sont comparables à l'état de l'art (Aral et al. 92%), avec un modèle plus léger (11M paramètres vs des CNN personnalisés souvent plus lourds). Le modèle offre un excellent compromis performance/efficacité, le rendant adapté au déploiement sur des machines modestes.

CHAPITRE 7 Démonstration du Modèle

7.1 Application de Démonstration : Eco-Tri

Une application interactive a été développée avec **Streamlit** pour permettre aux utilisateurs de tester le modèle sans connaissances techniques.

7.2 Architecture de l'Application

```
app/  
|-- app.py      # Interface utilisateur (Streamlit)  
|-- model.py    # Chargement et inférence du modèle  
|-- utils.py    # Prétraitement et multilingue
```

7.3 Fonctionnalités de l'Interface

7.3.1 Téléversement d'Image

L'utilisateur peut uploader une image (JPG, JPEG, PNG). L'image est automatiquement redimensionnée et normalisée selon les mêmes transformations que celles utilisées lors de l'entraînement (Resize 256, CenterCrop 224, normalisation ImageNet).

7.3.2 Prédiction en Temps Réel

Le modèle analyse l'image et retourne :

- La classe prédite avec le pourcentage de confiance.
- Un conseil de tri adapté à la catégorie (en français ou malgache).
- Un histogramme des probabilités pour les 5 classes.
- Le Top 3 des prédictions avec barres de progression.

7.3.3 Support Multilingue

TABLE 7.1 – Traductions français/malgache des classes

Classe	Français	Malagasy
cardboard	Carton	Kartôna
metal	Métal	Vy
paper	Papier	Taratasy
plastic	Plastique	Plastika
trash	Déchet non recyclable	Fako tsy azo averina

7.3.4 Visualisation des Résultats

L'application affiche également les courbes d'apprentissage et la matrice de confusion générées lors de l'entraînement, permettant à l'utilisateur de comprendre les performances du modèle.

La figure 7.1 présente différentes captures d'écran de l'interface utilisateur.

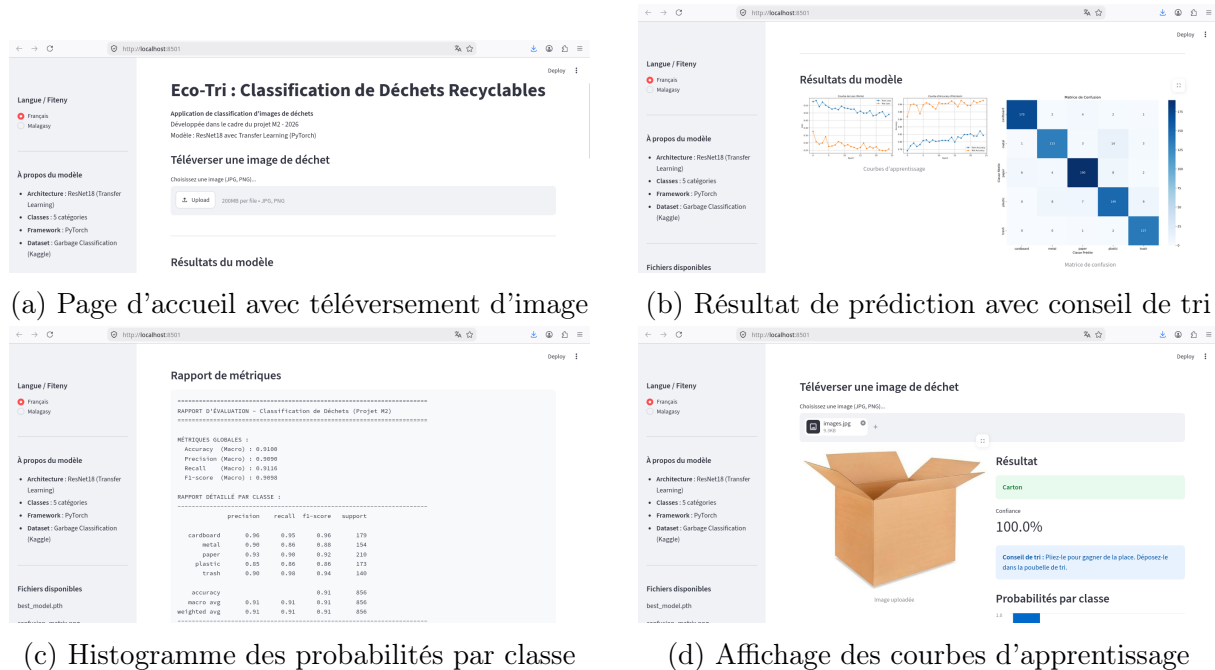


FIGURE 7.1 – Captures d'écran de l'application Eco-Tri

7.4 Guide d'Utilisation

1. Lancer l'application : `streamlit run app/app.py`
2. Sélectionner la langue (Français / Malagasy) dans la barre latérale.
3. Téléverser une image de déchet.
4. Visualiser instantanément le résultat de la classification.

7.5 Lien vers le Dépôt Git

Le code source complet, commenté et documenté, est disponible sur :

https://github.com/Maminiaina0601/eco_tri.git

Le dépôt contient :

- `app/` : Code source de l'application (Python).
- `colab/` : Notebook d'entraînement Jupyter.
- `data/` : Documentation du dataset.
- `results/` : Modèle entraîné et résultats.

- `requirements.txt` : Dépendances.
- `README.md` : Documentation complète.

CHAPITRE 8 Conclusion et Perspectives

8.1 Conclusion Générale

Ce projet a démontré la faisabilité d'un système de classification automatique de déchets recyclables utilisant le Deep Learning. Les contributions principales sont :

1. **Étude bibliographique** : Synthèse des fondamentaux du Deep Learning et des architectures CNN.
2. **Revue des travaux existants** : Analyse comparative des approches de classification de déchets.
3. **Implémentation Python** : Pipeline complet d'entraînement et d'inférence avec PyTorch.
4. **Jeu de données documenté** : Dataset Kaggle de 4 272 images, 5 classes, avec prétraitement et augmentation.
5. **Résultats** : 91% d'accuracy, F1-score macro de 0.91, matrice de confusion détaillée.
6. **Démonstration** : Application Streamlit interactive bilingue (FR/MG).
7. **Code source** : Dépôt Git avec code commenté et documentation.

8.2 Forces de l'Approche

- Performance élevée (91%) avec un modèle léger (11M paramètres).
- Reproductibilité assurée par le pipeline documenté.
- Application accessible avec interface interactive.
- Impact sociétal via la sensibilisation au tri sélectif.

8.3 Limitations

- 5 classes seulement (verre exclu).
- Légers déséquilibres dans les données.
- Surapprentissage modéré après 20 époques.

8.4 Perspectives

1. **Dataset** : Ajouter des classes (verre, organique, électronique), collecter plus d'images.
2. **Modèle** : Tester EfficientNet, ConvNeXt, Vision Transformers.
3. **Augmentation** : MixUp, CutMix, GANs pour générer des variations.
4. **Déploiement mobile** : Conversion ONNX ou TensorFlow Lite pour smartphone.

5. **Détection** : Passer de la classification à la détection d'objets (YOLO).
6. **Continuous Learning** : Boucle de rétroaction pour amélioration continue.

Bibliographie

- [1] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 86(11) :2278-2324, 1998.
- [2] K. He, X. Zhang, S. Ren et J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [3] D. P. Kingma et J. Ba, *Adam : A Method for Stochastic Optimization*, International Conference on Learning Representations (ICLR), 2015.
- [4] A. Krizhevsky, I. Sutskever et G. E. Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, Advances in Neural Information Processing Systems (NeurIPS), 2012.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li et L. Fei-Fei, *ImageNet : A Large-Scale Hierarchical Image Database*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2009.
- [6] M. Abal, *Garbage Classification Dataset*, Kaggle, 2020. <https://www.kaggle.com/datasets/mostafaabal/garbage-classification>
- [7] M. Yang et G. Thung, *Classification of Trash for Recyclability Status*, CS229 Project Report, Stanford University, 2016.
- [8] R. A. Aral, S. R. Keskin, M. Kaya et M. Hacıömeroğlu, *Classification of TrashNet Dataset Based on Deep Learning Models*, IEEE International Conference on Big Data, 2018.

CHAPITRE A Guide d'Installation et d'Exécution

A.1 Prérequis

- Python 3.10 ou supérieur
- pip (gestionnaire de paquets)
- Git

A.2 Installation

```
1 # Cloner le depot
2 git clone https://github.com/Maminiaina0601/eco_tri.git
3 cd eco-tri
4
5 # Installer les dependances
6 pip install -r requirements.txt
7
8 # Lancer l'application
9 streamlit run app/app.py
```

Listing A.1 – Commandes d'installation

A.3 Entraînement du Modèle (Google Colab)

1. Ouvrir colab/training.ipynb sur Google Colab.
2. Runtime → Changer le type → GPU T4.
3. Exécuter toutes les cellules (15-20 min).
4. Télécharger les fichiers dans `results/` :
 - `best_model.pth`
 - `training_curves.png`
 - `confusion_matrix.png`
 - `metrics_report.txt`

A.4 Redémarrer l'Application

```
1 # Apres avoir place le modele dans results/
2 streamlit run app/app.py
```